MERGEFLOW - LIVE DEMO & VIVA GUIDE

Academic Project Guide for ISA Viva & Demonstration

Group G1: Merge Sort + Recursion Tree Method

Target Audience: Third Year Computer Engineering Examiner / Professor

Repository: https://github.com/PixelNinja420/Merge-Sort

1. Fast 2-3 Minute Demonstration Script

Follow this exact sequence during your live presentation for maximum impact:

Step 1: Launch & Introduction (30 seconds)

"Good morning, Professor. For our G1 ISA Mini-Project, we built MergeFlow, an educational and practical File a

>

Our desig

Step 2: Show the Numerical Example from Class Notes (45 seconds)

1. In the workbench, click the "PDF Example" button.

2. Point t

Step 3: Explain the Recursion Tree & Activity Feed (45 seconds)

1. Scroll down to the Recursion Tree Explorer.

2. Click t

Step 4: Demonstrate CSV Multi-Column Sorting & Stability (30 seconds)

1. Scroll up and click "Sample CSV".

2. Show

Step 5: Exporting & Conclusion (15 seconds)

1. Click "Download run report" or "Export sorted file".

"Finally, s

2. Key Explanations to Practice

How to Explain the Recursion Tree

"Professor, the recursion tree visually models the recurrence relation T(n) = 2T(n/2) + Theta(n). The root has

How to Explain the Comparison Counter

> *"Our comparison counter is not a static formula or an estimate. It is incremented directly inside the `while`*

3. Top 15 Viva Questions and Concise Answers

Q1: What is the divide-and-conquer strategy?

Answer: It is an algorithmic paradigm that breaks a problem into smaller subproblems of the same type, solves them recursively, and combines their solutions to form the solution to the original problem.

Q2: What is the recurrence relation for Merge Sort?

Answer: T(n) = 2T(n/2) + Theta(n) for n > 1, with T(1) = Theta(1). The 2T(n/2) represents sorting the two halves, and Theta(n) is the linear work of merging them.

Q3: What is the base case of Merge Sort?

Answer: The base case occurs when low >= high, meaning the subarray contains zero or one element. A single element is already sorted by definition, so the recursive call simply returns.

Q4: Why is mid calculated as floor((low + high) / 2)?

Answer: This evenly partitions the subarray into two halves of sizes ceil(n/2) and floor(n/2), ensuring balanced subtrees and guaranteeing a recursion depth of O(\log n).

Q5: What is the best, average, and worst-case time complexity of Merge Sort?

Answer: All three cases are O(n \log n). This is because Merge Sort unconditionally divides the array into halves and always performs linear merging work regardless of initial element ordering.

Q6: What is the auxiliary space complexity of Merge Sort?

Answer: O(n). Merging two sorted halves requires a temporary buffer or array to hold elements before copying them back to the original array.

Q7: What does "stable sorting" mean, and is Merge Sort stable?

Answer: A sorting algorithm is stable if records with identical sorting keys appear in the output in the same relative order as in the input. Merge Sort is stable because when L[i] == R[j], we select L[i] from the left half first (cmp <= 0).

Q8: How many recursive calls are made for an array of n elements?

Answer: Exactly 2n - 1 recursive calls (for n >= 1), corresponding to the total number of nodes in a full binary recursion tree with n leaves. For our 10-element PDF example, 2(10) - 1 = 19 calls.

Q9: How many merge operations occur for an array of n elements?

Answer: Exactly n - 1 merge operations. Each merge combines two segments into one, reducing the number of distinct segments from n to 1. For n = 10, there are 10 - 1 = 9 merges.

Q10: What is the maximum recursion depth for an array of size n?

Answer: ceil(log2 n). For n = 10, log2(10) ~ 3.32, so the maximum depth is 4 levels (levels 0, 1, 2, 3, 4).

Q11: Why is Merge Sort preferred for sorting linked lists and large external files?

Answer:

1. For lin

Q12: How does your tool handle CSV rows with multiple columns?

Answer: The CSV parser extracts rows into arrays of cell values. When the user selects a sort column, that cell is used as the key, but the entire row array is swapped and moved as an atomic unit, preventing data corruption.

Q13: How does your tool handle commas inside quoted CSV fields?

Answer: It uses an RFC-4180 state machine. When inside an unclosed double quote ("), commas and line breaks are treated as literal text instead of field or row delimiters.

Q14: How does your tool handle timestamp sorting?

Answer: It parses timestamp strings into epoch milliseconds using Date.parse(). If parsing fails, it gracefully falls back to natural alphanumeric comparison so that no data is lost or displaced.

Q15: Why can Quick Sort be faster than Merge Sort in practice despite worse worst-case complexity?

Answer: Quick Sort sorts in place with O(1) auxiliary memory and has superior CPU cache locality with smaller constant factors, whereas Merge Sort requires allocating and copying memory to temporary arrays.